WhatsApp AI Agent — Detailed Implementation Plan

From Local Workflow to Production

Overview

This document is the step-by-step technical implementation plan. It is organized into phases, each building on the previous. Every phase ends with a working, testable deliverable.

Phase 1 — Core Workflow (Local)

Goal: A working n8n workflow that receives a message, processes it through an LLM, and returns a reply. No database, no MCP, no WhatsApp yet — just the agent loop working end to end locally.

Step 1.1 — Create the Master Workflow in n8n

Open n8n at http://localhost:5678 and create a new workflow named: WhatsApp AI Agent - Master

---

Step 1.2 — Webhook Node (Message Entry Point)

This node receives all incoming messages. Later it will receive real WhatsApp webhooks from Meta. For now it acts as a simulator.

Configuration:

- Node type: Webhook
- HTTP Method: POST
- Path: whatsapp
- Response Mode: Using Respond to Webhook Node
- Authentication: None (add later in production)

Step 1.3 — Function Node (Parse Incoming Payload)

This node extracts the relevant fields from the incoming webhook body. It standardizes the data so the rest of the workflow always receives a consistent structure regardless of where the

message came from.

Configuration:

- Node type: Code (JavaScript)
- Operation: Extract user_phone, business_number, message_text from the incoming body
- Output: A clean object with those three fields plus a timestamp

Logic description:
Read the incoming JSON body. Extract the message text from body.message, the sender phone from body.from, and the business number from body.to. If any field is missing, set a default. Output a single item with all three fields normalized.

Step 1.4 — AI Agent Node (LLM Brain)

This is the core of the workflow. It takes the message and decides what to do — reply directly or call a tool.

Configuration:

- Node type: AI Agent
- Prompt source: Define below
- Prompt: ={{ $json.message_text }}
- System prompt: You are a helpful assistant for a Lebanese business. Reply in the same language the customer uses (Arabic, French, or English). Be concise, friendly, and professional. If you do not know the answer, say so and offer to connect the customer with a human.
- Memory: Window Buffer Memory node attached (see Step 1.5)
- Tools: None yet (added in Phase 3)

Step 1.5 — Window Buffer Memory Node

Attached to the AI Agent node. Keeps conversation context so the agent remembers what was said earlier in the same session.

Configuration:

- Node type: Window Buffer Memory
- Session ID: ={{ $json.user_phone }}
- Context Window Length: 10 messages

This means each unique phone number gets its own conversation context stored in n8n's internal memory during the session.

Step 1.6 — Groq Chat Model Node

Attached to the AI Agent node as the language model.

Configuration:

- Node type: Groq Chat Model
- API Key: your Groq API key from console.groq.com
- Model: llama-3.3-70b-versatile
- Temperature: 0.3 (low for consistent, professional replies)
- Max Tokens: 500

Step 1.7 — Respond to Webhook Node (Reply Sender)

Sends the agent's reply back to whoever called the webhook.

Configuration:

- Node type: Respond to Webhook
- Response Code: 200
- Response Body: ={{ $json.output }}

Step 1.8 — Test the Workflow Locally

Activate the workflow and test using curl from PowerShell:

curl -X POST http://localhost:5678/webhook-test/whatsapp -H "Content-Type: application/json" -d "{"message_text": "Hello, what are your opening hours?", "from": "96170000001", "to": "96171000002"}"

Expected result: A JSON response containing a natural language reply from the LLM.

Deliverable: Agent loop working locally. Input a message, get an intelligent reply back.

Phase 2 — FastAPI MCP Server

Goal: Build the backend that handles all database operations. n8n will call this server to load

client configs, save conversations, and manage business data. This is what enables multi-tenancy.

Step 2.1 — Project Structure

Create a folder called agent-api with the following structure:

agent-api/
main.py (FastAPI app entry point)
database.py (PostgreSQL connection and session)
models.py (SQLAlchemy ORM models)
schemas.py (Pydantic request/response schemas)
routers/
clients.py (client config endpoints)
conversations.py (conversation log endpoints)
orders.py (order management endpoints)
appointments.py (appointment endpoints)
.env (environment variables)
requirements.txt

Step 2.2 — Dependencies

requirements.txt contents:
fastapi
uvicorn
sqlalchemy
psycopg2-binary
redis
python-dotenv
pydantic

Step 2.3 — Database Models

Five tables as defined in the system design document:

businesses table: id, name, industry, system_prompt, enabled_tools (JSON array), escalation_number, subscription_plan, subscription_status, created_at

whatsapp_numbers table: id, business_id (foreign key), phone_number (unique), display_name, is_active

conversations table: id, business_id (foreign key), customer_phone, role (user or assistant), message, created_at

orders table: id, business_id (foreign key), customer_phone, customer_name, items (JSON), status, notes, created_at

appointments table: id, business_id (foreign key), customer_phone, customer_name, scheduled_at, status, notes, created_at

---

Step 2.4 — API Endpoints to Build

Client endpoints:

- GET /client/config/{business_id} — returns full client config row from businesses table
- GET /client/by-phone/{phone_number} — looks up whatsapp_numbers table and returns the associated business_id

Conversation endpoints:

- POST /conversation/log — inserts one message row (role, message, business_id, customer_phone)
- GET /conversation/history/{business_id}/{customer_phone} — returns last 10 messages ordered by created_at ascending

Order endpoints:

- POST /order/create — inserts new order row
- GET /order/{order_id} — returns single order
- PUT /order/{order_id}/status — updates order status field only

Appointment endpoints:

- POST /appointment/create — inserts new appointment row
- GET /appointment/{business_id}/upcoming — returns all appointments with scheduled_at in the future ordered by date

---

Step 2.5 — Environment Variables

.env file contents:
DATABASE_URL=postgresql://user:password@localhost:5432/agentdb
REDIS_URL=redis://localhost:6379
SECRET_KEY=your_secret_key_here

Step 2.6 — Run PostgreSQL Locally for Testing

Use Docker to run PostgreSQL locally without installing it:

docker run -d --name agentdb -e POSTGRES_USER=user -e POSTGRES_PASSWORD=password -e POSTGRES_DB=agentdb -p 5432:5432 postgres:15

Then run the FastAPI server:

uvicorn main:app --reload --port 8000

API docs auto-generated at: http://localhost:8000/docs

Step 2.7 — Seed One Test Client

Insert a test business row directly into the businesses table:

```
INSERT INTO businesses (name, industry, system_prompt, enabled_tools, escalation_number, subscription_plan)
VALUES (
'Test Clinic',
'clinic',
'You are the virtual assistant for Test Clinic. You handle appointment bookings and answer questions about the clinic. The clinic is open Monday to Saturday 9am to 6pm. Always reply in the same language the customer uses.',
'["appointments", "faq"]',
'96170000001',
'business'
);
```

Then insert the WhatsApp number mapping for that business.

Step 2.8 — Connect n8n to FastAPI

Go back to the n8n workflow and add two HTTP Request nodes before the AI Agent node:

Node A — Resolve business_id:

- Method: GET
- URL: http://localhost:8000/client/by-phone/{{ $json.business_number }}
- Output: business_id

Node B — Load client config:

- Method: GET
- URL: http://localhost:8000/client/config/{{ $json.business_id }}
- Output: system_prompt, enabled_tools, escalation_number

Update the AI Agent system prompt to use the dynamically loaded value:

- System Prompt: ={{ $json.system_prompt }}

Now every client gets their own system prompt loaded at runtime. Multi-tenancy is solved.

Step 2.9 — Save Conversations to DB

After the AI Agent node, add two more HTTP Request nodes:

Node — Log user message:

- Method: POST
- URL: http://localhost:8000/conversation/log
- Body: business_id, customer_phone, role=user, message=the incoming message

Node — Log assistant reply:

- Method: POST
- URL: http://localhost:8000/conversation/log
- Body: business_id, customer_phone, role=assistant, message=the agent output

Deliverable: Full multi-tenant agent loop with persistent conversation logging. Different phone numbers map to different businesses with different system prompts.

Phase 3 — Tool Integration

Goal: Give the agent real tools it can use — appointment booking, order taking, escalation to human. This is what turns it from a chatbot into an agent.

Step 3.1 — Appointment Booking Tool

Add an HTTP Request tool node to the AI Agent node:

Tool name: book_appointment Description: Use this tool when the customer wants to book an appointment. Collect their name, preferred date and time, and the service they want. Then call this tool. Method: POST URL: http://localhost:8000/appointment/create Body fields: business_id, customer_phone, customer_name, scheduled_at, notes

The agent will call this tool automatically when it detects booking intent in the customer message.

Step 3.2 — Order Taking Tool

Tool name: create_order Description: Use this tool when the customer wants to place an order. Collect all order items, quantities, delivery address, and payment method. Then call this tool. Method: POST URL: http://localhost:8000/order/create Body fields: business_id, customer_phone, customer_name, items (JSON array), notes

---

Step 3.3 — Order Status Tool

Tool name: get_order_status Description: Use this tool when the customer asks about the status of their order. Method: GET URL: http://localhost:8000/order/{order_id}

Step 3.4 — Human Escalation Tool

Tool name: escalate_to_human Description: Use this tool when you cannot handle the customer's request, when the customer is frustrated, or when they explicitly ask to speak to a human. Method: POST URL: http://localhost:8000/escalation/create (add this endpoint to FastAPI)

What this endpoint does: logs the escalation event, then sends a WhatsApp message to the business owner's escalation_number via Meta Cloud API notifying them that a customer needs attention, including the customer's phone number and last message.

Step 3.5 — After-Hours Detection

Add a Function node before the AI Agent node that checks the current time in Beirut timezone (UTC+3). If the current time is outside business hours (defined in the client config), override the system prompt to include: "The business is currently closed. Inform the customer of the business hours and offer to take their request for the team to follow up when they open."

Step 3.6 — Google Calendar Integration (Optional for Clinics/Salons)

If the client uses Google Calendar for appointment management:

Add a Google Calendar node to the workflow as a tool:

- Operation: Create Event
- Calendar: the client's Google Calendar ID (stored in their businesses config)
- Title: Appointment with {customer_name}
- Start time: scheduled_at from the booking
- Description: customer_phone and service notes

This requires the client to connect their Google account to your n8n instance via OAuth. n8n handles this natively.

Deliverable: A fully functional AI agent that can book appointments, take orders, check order status, escalate to humans, and handle after-hours — all triggered automatically by the LLM based on customer intent.

Phase 4 — WhatsApp Integration (Meta Cloud API)

Goal: Connect the agent to a real WhatsApp number so actual customers can message it.

Step 4.1 — Set Up Meta Developer Account

1. Go to developers.facebook.com and create an account
2. Create a new App — choose Business type
3. Add the WhatsApp product to your app
4. Go to WhatsApp > API Setup

Step 4.2 — Get a Test Phone Number

Meta provides a free test number by default. You get 5 free test conversations per day without needing a real business number. This is enough for development and demos.

Note the following values from the API Setup page:

- Phone Number ID

- WhatsApp Business Account ID
- Temporary Access Token (valid 24h) or generate a permanent System User Token

---

Step 4.3 — Configure the Webhook

For Meta to send messages to your n8n, your n8n must be publicly accessible. Locally this requires a tunnel.

Install ngrok: https://ngrok.com/download

Run: ngrok http 5678

This gives you a public URL like https://abc123.ngrok.io

In Meta Developer Console:

- Webhook URL: https://abc123.ngrok.io/webhook/whatsapp
- Verify Token: any string you choose (e.g. my_verify_token_123)
- Subscribe to: messages

Step 4.4 — Handle Meta Webhook Verification

Meta sends a GET request to your webhook URL to verify it. You need to handle this in n8n.

Add a second workflow (or add a branch to the existing one) that:

- Detects GET requests to /webhook/whatsapp
- Reads the hub.verify_token query parameter
- If it matches your verify token, responds with hub.challenge query parameter value
- This is a one-time verification step

Step 4.5 — Parse Real Meta Webhook Payload

The real Meta webhook payload structure is different from your test payload. Update the Function node (Step 1.3) to parse the actual Meta format:

The real payload comes as: entry[0].changes[0].value.messages[0]
From this object extract:

- message text: .text.body
- sender phone: .from

- business number: entry[0].changes[0].value.metadata.display_phone_number

---

Step 4.6 — Send Reply via Meta Cloud API

Replace the Respond to Webhook node with an HTTP Request node that calls the Meta API:

Method: POST URL: https://graph.facebook.com/v18.0/{PHONE_NUMBER_ID}/messages Headers: Authorization: Bearer {ACCESS_TOKEN}, Content-Type: application/json Body: messaging_product: whatsapp to: {customer_phone} type: text text.body: {agent_reply}

Store the Phone Number ID and Access Token as n8n credentials (use the n8n Credentials Manager, not hardcoded values).

Step 4.7 — Test with Real WhatsApp

Send a WhatsApp message to your test number from any phone. You should receive an AI reply within seconds.

Deliverable: A real WhatsApp number that customers can message and receive intelligent AI replies from.

Phase 5 — VPS Deployment (Production)

Goal: Move everything from local to a publicly hosted VPS so it runs 24/7 without your laptop.

Step 5.1 — Provision Hetzner VPS

1. Create account at hetzner.com
2. Create new server: CX22 plan (2 vCPU, 4GB RAM, €4/month)
3. Choose Ubuntu 24.04 LTS
4. Add your SSH public key during setup
5. Note the server IP address

Step 5.2 — Initial Server Setup

SSH into the server: ssh root@YOUR_SERVER_IP

Run these commands in order:

Update system packages
Install Docker and Docker Compose plugin
Create a non-root user with sudo privileges
Set up a basic firewall allowing only ports 22 (SSH), 80 (HTTP), and 443 (HTTPS)

Step 5.3 — Get a Domain

Purchase a domain (e.g. yourproduct.com) from Namecheap or Cloudflare (~$10/year).

Create two DNS A records pointing to your VPS IP:

- app.yourproduct.com → YOUR_SERVER_IP (for n8n)
- api.yourproduct.com → YOUR_SERVER_IP (for FastAPI)

Wait for DNS propagation (5–30 minutes).

Step 5.4 — Docker Compose Production Stack

Create a folder on the VPS at /opt/agent/ containing:

docker-compose.yml defining five services: n8n, fastapi, postgres, redis, nginx

n8n service environment variables:

- N8N_HOST: app.yourproduct.com
- WEBHOOK_URL: https://app.yourproduct.com
- N8N_PROTOCOL: https
- DB_TYPE: postgresdb (point n8n to the same postgres instance)

FastAPI service:

- Built from your local ./api directory pushed to the server
- DATABASE_URL pointing to postgres service
- REDIS_URL pointing to redis service

PostgreSQL service with persistent volume and strong password

Redis service with persistent volume

Nginx service handling SSL termination and reverse proxying:

- app.yourproduct.com → n8n on port 5678
- api.yourproduct.com → FastAPI on port 8000

Step 5.5 — SSL Certificates

Install Certbot on the VPS and run it to get free Let's Encrypt SSL certificates for both subdomains. Certbot will automatically renew them every 90 days.

Step 5.6 — Deploy FastAPI Code

Option A: Copy the agent-api folder to the VPS via SCP and build the Docker image there.
Option B: Push your code to a private GitHub repo and pull it on the VPS.

Recommended for solo developer: Option B. Set up a simple deploy script that pulls the latest code and restarts the FastAPI container.

Step 5.7 — Start the Stack

On the VPS run: docker compose up -d

Verify all five containers are running: docker compose ps

Check n8n is accessible at https://app.yourproduct.com Check FastAPI docs at https://api.yourproduct.com/docs

---

Step 5.8 — Update Meta Webhook URL

Go back to Meta Developer Console and update the webhook URL from the ngrok URL to: https://app.yourproduct.com/webhook/whatsapp

Your workflow now runs on the VPS 24/7 without needing your laptop.

Step 5.9 — Import n8n Workflows

In n8n on the VPS, import your workflow JSON files that you built and tested locally. Re-enter the API credentials (Groq key, Meta token) in the n8n Credentials Manager.

Activate the workflow.

Deliverable: Fully hosted production system. Customers message your WhatsApp number, the agent handles it 24/7, all data stored in PostgreSQL on the VPS.

Phase 6 — Onboarding & Multi-Client Management

Goal: A repeatable process to onboard new clients in under 2 hours without touching code.

Step 6.1 — Client Onboarding Form

Build a simple form (Google Forms or Tally.so — both free) that collects:

- Business name and industry
- Business description (2–3 sentences)
- Services or products offered (list)
- Business hours
- Frequently asked questions (up to 10)
- Escalation WhatsApp number
- WhatsApp Business number to connect

Step 6.2 — Generate System Prompt

From the form responses, manually write the client's system prompt using this template:

You are the virtual assistant for {business_name}, a {industry} in Lebanon. {business_description}. You help customers with: {services_list}. Business hours are {hours}. Always reply in the same language the customer uses (Arabic, French, or English). If you cannot help, escalate to a human.

Then add the FAQ answers directly into the system prompt as a reference section.

Step 6.3 — Insert Client into Database

Run one INSERT statement into the businesses table with the generated system prompt and config. Then insert their WhatsApp number into the whatsapp_numbers table.

No code changes. No n8n workflow changes. The existing workflow picks up the new client automatically.

Step 6.4 — Connect Client WhatsApp Number

The client needs to connect their WhatsApp Business number to your Meta app. Two options:

Option A (simpler): The client creates their own Meta Business App and adds your webhook URL. They manage their own WhatsApp API access.

Option B (better UX): You create a WhatsApp Business Solution Provider setup where clients connect through your app via OAuth. More complex to set up but seamless for clients.

Start with Option A for the first few clients.

Step 6.5 — Test Before Going Live

Send 5–10 test messages covering: FAQ question, appointment booking request, order request, after-hours message, escalation trigger. Verify all scenarios work correctly before giving the client access.

Deliverable: New client onboarded in under 2 hours. Zero code changes per client. Fully repeatable process.

Phase 7 — Admin Dashboard (Basic)

Goal: A simple internal dashboard for you to monitor all clients, view conversation logs, and manage configs. Not client-facing yet.

Step 7.1 — Tech Stack

Next.js 14 (you already know it) + Tailwind CSS + shadcn/ui components. Connects directly to your FastAPI backend via REST.

Step 7.2 — Pages to Build

Clients list page: shows all businesses, their subscription plan, status, and last active time.

Client detail page: shows client config, WhatsApp number, system prompt, and ability to edit.

Conversations page: shows all conversations for a client, filterable by date and customer phone.

Escalations page: shows all escalation events requiring human follow-up.

Analytics page (basic): total messages handled today, this week, this month per client. Escalation rate. Most common message topics.

Step 7.3 — Authentication

Basic auth or a simple JWT login for yourself. This is an internal tool — it does not need to be multi-user yet.

Step 7.4 — Deploy Dashboard

Add a third subdomain: dashboard.yourproduct.com
Add the Next.js app as a new Docker service in your docker-compose.yml.
Add an Nginx proxy rule for the new subdomain.

Deliverable: You can manage all clients, read their conversations, and update configs from a browser instead of running SQL queries.

Summary: Phase Completion Checklist

Phase 1 — Local workflow: message in, LLM reply out. No DB.
Phase 2 — FastAPI MCP server: multi-tenant, conversation logging, client configs from DB.
Phase 3 — Tools: appointment booking, order taking, escalation, after-hours.
Phase 4 — WhatsApp: real Meta Cloud API integration, actual WhatsApp messages working.
Phase 5 — VPS: everything hosted, running 24/7, public domain, SSL.
Phase 6 — Onboarding: repeatable client onboarding process, first paying clients.
Phase 7 — Dashboard: internal management UI.

Estimated Timeline

Phase 1: 1–2 days
Phase 2: 3–5 days
Phase 3: 3–5 days
Phase 4: 1–2 days
Phase 5: 1–2 days
Phase 6: Ongoing (per client)
Phase 7: 1–2 weeks (can be done in parallel after Phase 5)

Total to first paying client: 2–3 weeks of focused work.